

2026 腾讯游戏安全技术竞赛 - PC客户端安全 - 初赛 Writeup - Li Jieheng

本文档包含完整的分析过程、关键截图、算法说明、所有挫折和纠错记录。

题目概述

穿越火线ACE安全团队截获外挂组织「影（Shadow）」的门禁系统「宫殿」。系统由用户态控制台（ShadowGateApp.exe）+ 内核驱动（ShadowGateSys.sys）组成，驱动内部隐藏加密迷宫。系统不会直接反馈操作结果，需通过侧信道获取信息。目标：突破迷宫，取出Flag。

评分（满分10分）

评分项	满分	状态
加载驱动并正确通信，理解基本机制	1.5	✅ 已完成
发现隐匿通信手段（侧信道），编写检测工具	5.0	✅ 已确认5/5种
还原完整迷宫地图 + 自动化探索脚本	1.5	✅ 已完成
最短路径求解	0.5	✅ 已完成
提交正确Flag	1.5	✅ flag{SHADOWNT_HYPERVMX}

一、环境搭建

- Windows 10 22H2 x64 虚拟机（VMware，BIOS模式）
- WinDbg内核调试已连接（串口 \\.\pipe\com_debug，debugport:2）
- 测试签名已开启（bcdedit /set testsigning on）
- VMware Tools已安装
- Visual Studio 2022（x64 Release配置）

二、驱动加载

- 驱动文件：ShadowGateSys.sys（4.01MB）

- 服务名： ShadowGate

- 加载命令：

```
sc create ShadowGate type= kernel binPath= C:\Users\CTF\Desktop\ShadowGateSys.sys
sc start ShadowGate
```

- 状态： RUNNING
-

三、静态逆向分析

驱动为x64，使用IDA Pro反编译：

- DriverEntry → security_cookie() → EntryPoint()
 - EntryPoint() 调用 ExAllocatePool2(64, 472, 'ezaM') 分配472字节NonPagedPool内存
 - 第一个参数64：表示不可分页内存
 - 第二个参数472：分配字节数
 - 第三个参数 1702519117： Pool Tag，转换成字符串是 "Maze"
 - 全局指针 P（IDA地址 0x1400050B8）指向该内存块，存储迷宫数据结构
 - 随后调用 sub_1400018A0 进入设备创建逻辑
 - sub_1400018A0 是thunk函数，跳转到 loc_1403FC0E2 ——**代码被重度混淆**（疑似VMProtect），含花指令、不透明谓词、垃圾数据，静态分析极其困难
-

四、导入函数分析（侧信道线索）

从IDA导入表发现以下关键API，指示至少4种侧信道：

导入函数	推测用途
ZwOpenEvent + ZwSetEvent	Event对象信号通知
KeReleaseSemaphore	Semaphore计数变化
KeDelayExecutionThread	时间延迟差异
ZwQueryVirtualMemory + PsGetProcessPeb	进程内存/PEB修改
KeStackAttachProcess + KeUnstackDetachProcess	附加目标进程修改内存

其他关键导入： IoCreateDevice 、 IoCreateSymbolicLink 、 ExAllocatePool2 、
ExFreePoolWithTag 、 ProbeForRead 、 ProbeForWrite 、 IoCompleteRequest 、
PsGetCurrentProcessId 、 PsGetCurrentThreadId 、 PsLookupProcessByProcessId 、
PsLookupThreadByThreadId 、 KeInitializeSpinLock 、 KeAcquireSpinLockRaiseToDpc 、
KeReleaseSpinLock 、 ObReferenceObjectByName

五、答疑情报（专家提示）

- 不需要做拦截，侧信道是被动可观测的
- 侧信道分层设计，不同层次有不同的泄露手段（用户态+内核态）
- 思路与反病毒/反外挂检测技术相通
- 可观测信道是故意设计的，一定存在
- 发现的侧信道种类越多，得分越高

六、WinDbg动态分析 — 派遣函数与设备名查找

6.1 派遣函数分析

通过 `kd> !drvobj \Driver\ShadowGate 2` 获取驱动的派遣函数表：

驱动只注册了3个有效的派遣函数：

IRP类型	函数地址	说明
IRP_MJ_CREATE (0x00)	fffff806949314b0	打开设备时触发
IRP_MJ_CLOSE (0x02)	fffff80694931410	关闭设备时触发
IRP_MJ_DEVICE_CONTROL (0x0e)	fffff80694931540	核心IOCTL处理

其他全是 nt!IopInvalidDeviceRequest , 不支持。

6.2 设备名查找过程

尝试一：!devobj 查询设备对象（失败）

```
1: kd> !devobj ffff85023e846e30
ffff85023e846e30: is not a device object
```

犯了一个错误—— ffff85023e846e30 是 Driver Object（驱动对象），而 !devobj 需要的是 Device Object（设备对象）地址，两者不同。

```
1: kd> !devobj ffff85023e846e30
ffff85023e846e30: is not a device object
```

```
1: kd> !devobj ffff85023e846e30
ffff85023e846e30: is not a device object
1: kd> !devnode 0 1 ShadowGate
Dumping IopRootDeviceNode (= 0xfffff850236edacb0)
```

尝试二：!devnode 查询PnP设备树（失败）

ShadowGate 是通过 sc create 手动加载的 legacy 驱动，不在PnP设备树中。

```
Directory Object: fffff8011b470c90 Name: ShadowGate
1: kd> !object \DosDevices\ShadowGate
Object DosDevices\ShadowGate not found
```

尝试三：!object 过滤查询（成功）

ffffce811bbae6e0	SymbolicLink	Tcp
ffffce811ba4cad0	SymbolicLink	Harddisk0Partition1
12 fffffce8122831800	SymbolicLink	ShadowGate
ffffce811f859ec0	SymbolicLink	USB#VID_0E0F&PID_0008#000650268328#{0850302a-b344-4fda-9be9-90576b8d46f0}
ffffce811e4941e0	SymbolicLink	ACPI#VMW0003#4&1bd7f811&0#{378de44c-56ef-11d1-bc8c-00a0c91405dd}
ffffce811e4949c0	SymbolicLink	ACPI#PNP0303#4&1bd7f811&0#{884b96c3-56ef-11d1-bc8c-00a0c91405dd}
ffffce811e4945d0	SymbolicLink	ROOT#SYSTEM#0000#{0a4252a0-7e70-11d0-a5d6-28db04c10000}
ffffce811ba4dcd0	SymbolicLink	Harddisk0Partition2
ffffce811b95db20	SymbolicLink	Scsi1:
13 fffffce811f853db0	SymbolicLink	NduIoDevice
ffffce811be9c180	SymbolicLink	PRN
ffffce811ba4dd60	SymbolicLink	Harddisk0Partition3
ffffce811b95d190	SymbolicLink	ROOT#spaceport#0000#{53f5630e-b6bf-11d0-94f2-00a0c91efb8b}

```
1: kd> !object \Device\ShadowGate
Object: fffff85023e7ace10  Type: (fffff850236ee2400) Device
  ObjectHeader: fffff85023e7acde0 (new version)
  HandleCount: 0  PointerCount: 2
  Directory Object: fffffce811b470c90  Name: ShadowGate
```

设备名确认为 \Device\ShadowGate 。

6.3 符号链接查找

\DosDevices\ShadowGate 不存在。通过遍历 \GLOBAL?? 目录找到符号链接。用户态可通过

\\.\ShadowGate 打开设备。

```
fffff80b 81006e40 cc          int     3
1: kd> !drvobj \Driver\ShadowGate 2
Driver object (fffff85023e846e30) is for:
  \Driver\ShadowGate

DriverEntry:  fffff80694938000
DriverStartIo: 00000000
DriverUnload: fffff80694931840
AddDevice:    00000000

Dispatch routines:
[00] IRP_MJ_CREATE                fffff806949314b0  +0xfffff806949314b0
[01] IRP_MJ_CREATE_NAMED_PIPE    fffff8068154a780  nt!IopInvalidDeviceRequest
[02] IRP_MJ_CLOSE                 fffff80694931410  +0xfffff80694931410
[03] IRP_MJ_READ                  fffff8068154a780  nt!IopInvalidDeviceRequest
[04] IRP_MJ_WRITE                 fffff8068154a780  nt!IopInvalidDeviceRequest
[05] IRP_MJ_QUERY_INFORMATION     fffff8068154a780  nt!IopInvalidDeviceRequest
[06] IRP_MJ_SET_INFORMATION       fffff8068154a780  nt!IopInvalidDeviceRequest
[07] IRP_MJ_QUERY_EA              fffff8068154a780  nt!IopInvalidDeviceRequest
[08] IRP_MJ_SET_EA                fffff8068154a780  nt!IopInvalidDeviceRequest
[09] IRP_MJ_FLUSH_BUFFERS         fffff8068154a780  nt!IopInvalidDeviceRequest
[0a] IRP_MJ_QUERY_VOLUME_INFORMATION fffff8068154a780  nt!IopInvalidDeviceRequest
[0b] IRP_MJ_SET_VOLUME_INFORMATION fffff8068154a780  nt!IopInvalidDeviceRequest
```

七、WinObjEx64 设备属性分析

Device Properties

Basic

Object

Type

Security

Basic Information



Name:

ShadowGate

Type:

Device

Attributes:

KernelObject

Paged Pool Charge:

0

References:

2

Nonpaged Pool Charge:

0

Handles:

0

Header:

0xFFFFF85023E7ACDE0

Object:

0xFFFFF85023E7ACE10

确定

取消

应用(A)

Device

Device Properties



Basic Object Type Security

Object Information

Field	Value	Additional Information
DEVICE_OBJECT		
Type	0x0003	IO_TYPE_DEVICE
Size	0x0150	
ReferenceCount	0	
DriverObject	0xFFFF85023E846E30	\Driver\ShadowGate
NextDevice	NULL	
AttachedDevice	NULL	
CurrentIrp	NULL	
Timer	NULL	PIO_TIMER
Flags	0x00000044	DO_BUFFERED_IO DO_DEVICE_HAS_NAME
Characteristics	0x00000100	FILE_DEVICE_SECURE_OPEN
Vpb	NULL	PVPB
DeviceExtension	NULL	
DeviceType	0x00000022	FILE_DEVICE_UNKNOWN
StackSize	1	
Queue		
AlignmentRequirement	0x00000000	FILE_BYTE_ALIGNMENT
DeviceQueue		
Dpc		
ActiveThreadCount	0	

确定

取消

应用(A)

Device Properties



Basic Object Type Security

Common Information



A Device is an object that represents a physical, logical, or virtual device.

Object Count:	482	Default Paged Pool Charge:	0	Flags:	
Peak Object Count:	484	Default Nonpaged Pool Charge:	424		
Handle Count:	0	Memory Pool Type For Objects:	NonPagedPoolNx		
Peak Handle Count:	3				

Invalid Attributes and Access Rights

Invalid Attributes

0x00000100
Valid Access
0x001F01FF
Generic Read
0x00120089
Generic Write
0x00120116
Generic Execute
0x001200A0
Generic All
0x001F01FF

Flag

OBJ_OPENLINK

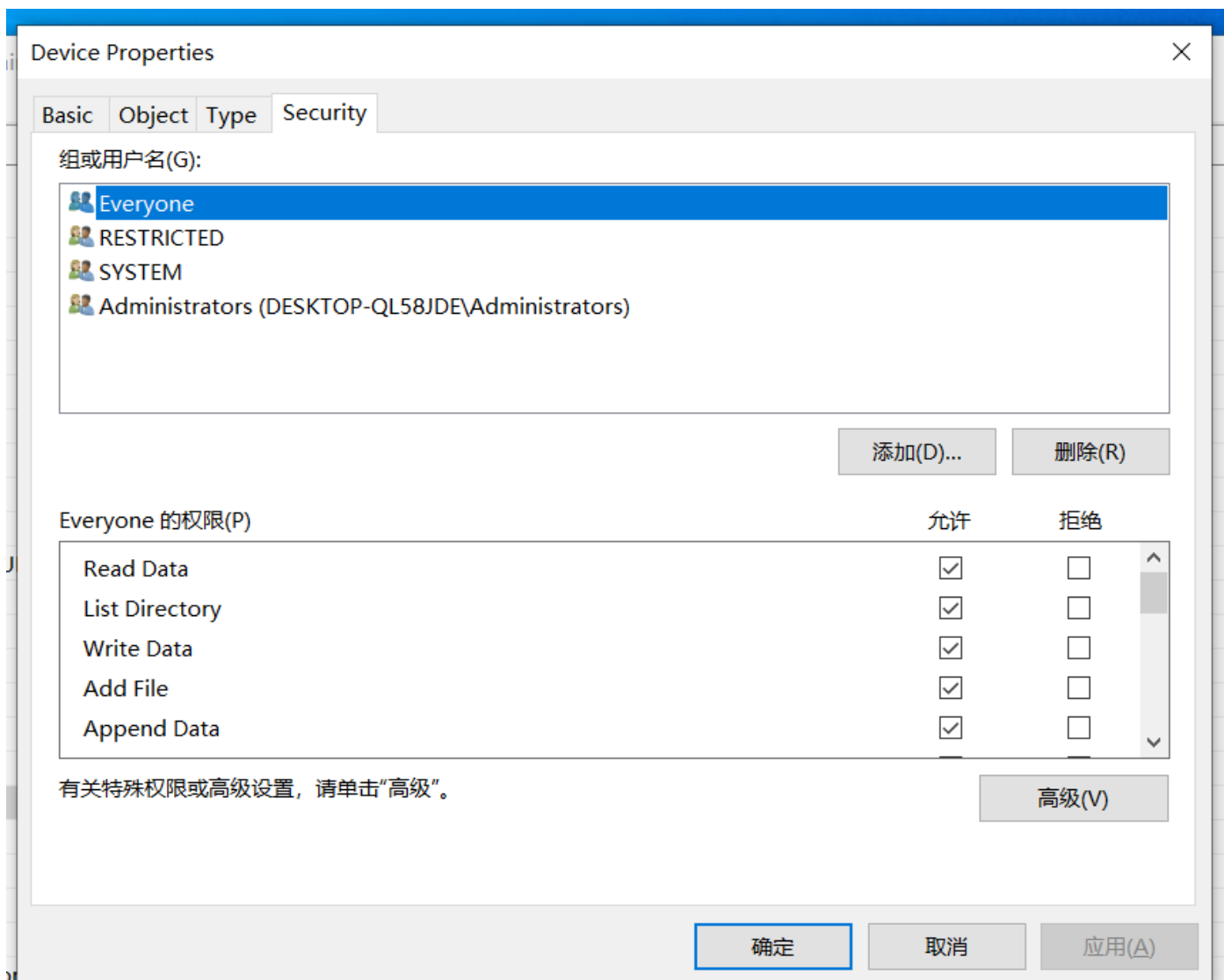
Value

0x00000100

确定

取消

应用(A)

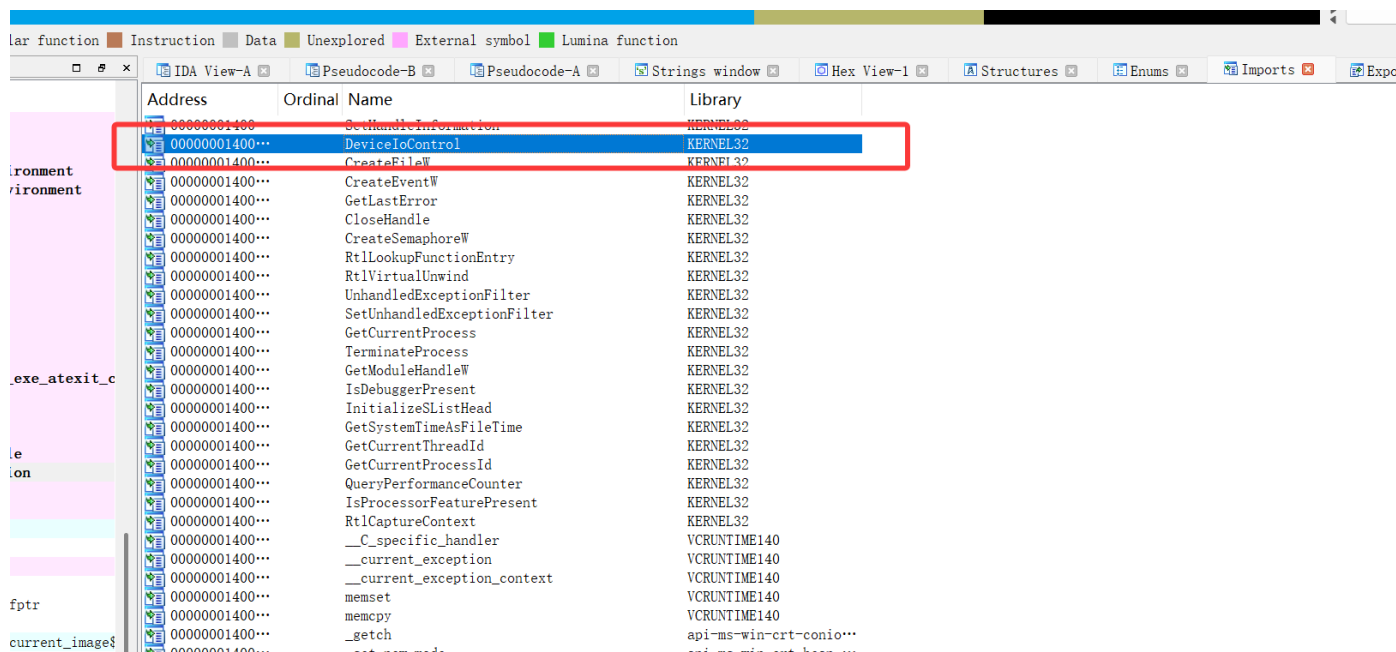


属性	值	说明
DeviceType	0x00000022	FILE_DEVICE_UNKNOWN
Flags	0x00000044	DO_BUFFERED_IO DO_DEVICE_HAS_NAME
DeviceExtension	NULL	状态全在全局指针P指向的472字节内存中
Characteristics	0x00000100	FILE_DEVICE_SECURE_OPEN
Security	Everyone RW	无需管理员即可通信

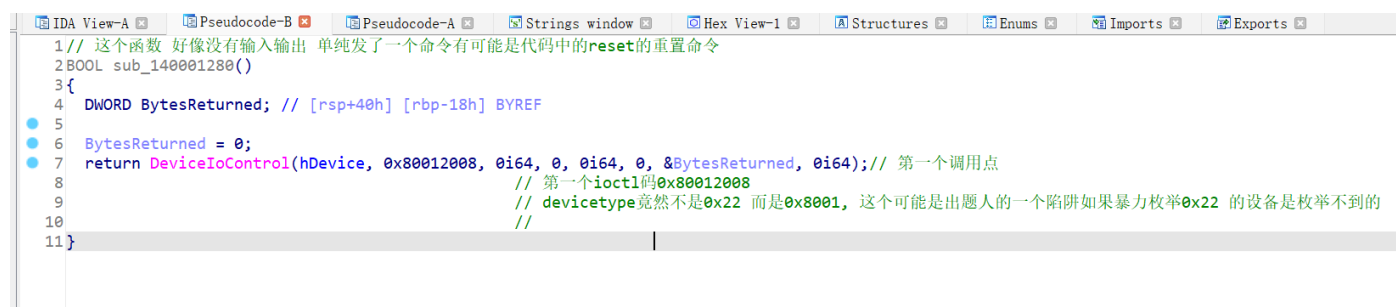
注意：WinObjEx64显示的DeviceType(0x22)与实际IOCTL码使用的DeviceType(0x8001)不同，这是出题人故意设置的干扰信息。

八、控制台程序逆向 — IOCTL码提取

在IDA中对 DeviceIoControl 做交叉引用，找到3个调用点。



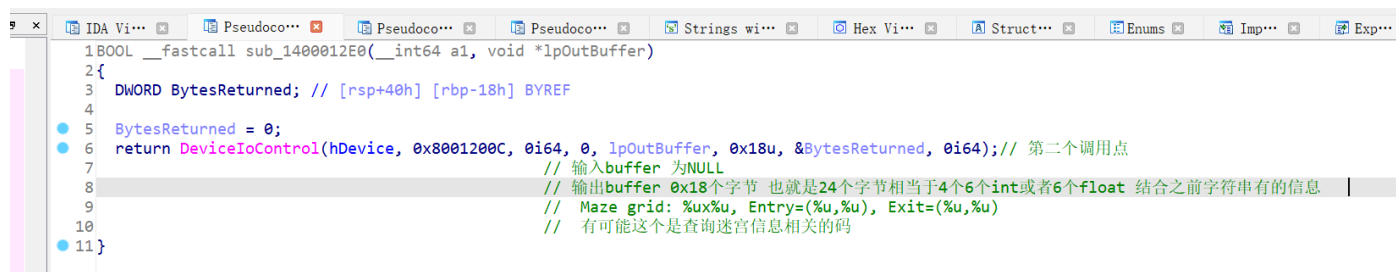
8.1 Reset (重置)



IOCTL码: 0x80012008

重要发现: DeviceType 不是 0x22 而是 0x8001 (自定义设备类型), 这就是之前扫描全部失败的原因。

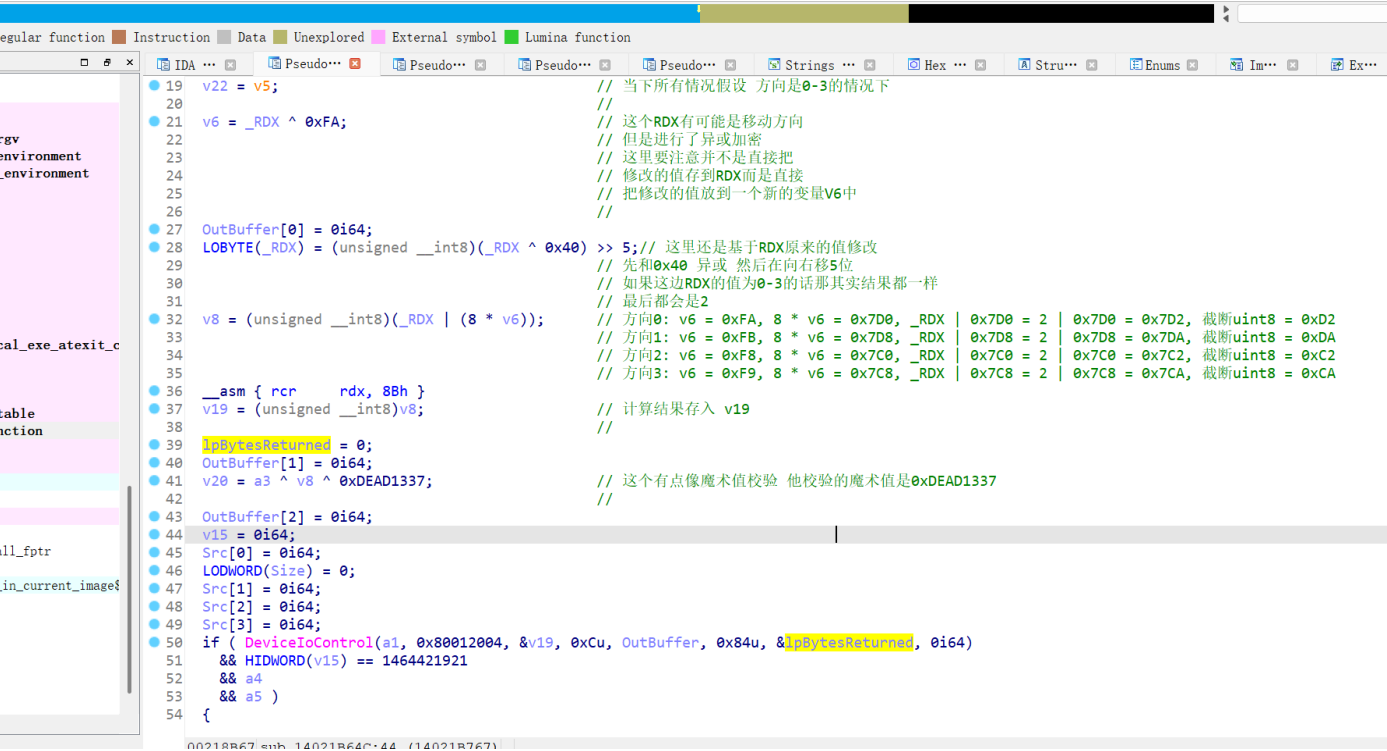
8.2 Query (查询迷宫信息)



IOCTL码: 0x8001200C

输出24字节 = 6个DWORD: 宽度、高度、入口X、入口Y、出口X、出口Y。

8.3 Move（移动指令）



```
19  v22 = v5; // 当下所有情况假设 方向是0-3的情况下
20
21  v6 = _RDX ^ 0xFA; // 这个RDX有可能是移动方向
22 // 但是进行了异或加密
23 // 这里要注意并不是直接把
24 // 修改的值存到RDX而是直接
25 // 把修改的值放到一个新的变量v6中
26
27  OutBuffer[0] = 0i64;
28  LOBYTE(_RDX) = (unsigned __int8)(_RDX ^ 0x40) >> 5; // 这里还是基于RDX原来的值修改
29 // 先和0x40 异或 然后在向右移5位
30 // 如果这边RDX的值为0-3的话那其实结果都一样
31 // 最后都会是2
32  v8 = (unsigned __int8)(_RDX | (8 * v6)); // 方向0: v6 = 0xFA, 8 * v6 = 0x7D0, _RDX | 0x7D0 = 2 | 0x7D0 = 0x7D2, 截断uint8 = 0xD2
33 // 方向1: v6 = 0xFB, 8 * v6 = 0x7D8, _RDX | 0x7D8 = 2 | 0x7D8 = 0x7DA, 截断uint8 = 0xDA
34 // 方向2: v6 = 0xF8, 8 * v6 = 0x7C0, _RDX | 0x7C0 = 2 | 0x7C0 = 0x7C2, 截断uint8 = 0xC2
35 // 方向3: v6 = 0xF9, 8 * v6 = 0x7C8, _RDX | 0x7C8 = 2 | 0x7C8 = 0x7CA, 截断uint8 = 0xCA
36  __asm { rcr     rdx, 8Bh }
37  v19 = (unsigned __int8)v8; // 计算结果存入 v19
38
39  lpBytesReturned = 0;
40  OutBuffer[1] = 0i64;
41  v20 = a3 ^ v8 ^ 0xDEAD1337; // 这个有点像魔术值校验 他校验的魔术值是0xDEAD1337
42
43  OutBuffer[2] = 0i64;
44  v15 = 0i64;
45  Src[0] = 0i64;
46  LODWORD(Size) = 0;
47  Src[1] = 0i64;
48  Src[2] = 0i64;
49  Src[3] = 0i64;
50  if ( DeviceIoControl(a1, 0x80012004, &v19, 0xCu, OutBuffer, 0x84u, &lpBytesReturned, 0i64)
51      && HIDWORD(v15) == 1464421921
52      && a4
53      && a5 )
54  {
```

IOCTL码： 0x80012004

输入12字节（编码后的方向+校验值），输出132字节。

8.4 IOCTL码汇总

IOCTL	功能	输入	输出
0x80012004	移动	12字节	132字节
0x80012008	重置	无	无
0x8001200C	查询	无	24字节

九、控制台运行与基本信息确认

```
管理员: 命令提示符 - C:\Users\CTF\Desktop\ShadowGateApp.exe

ACE has intercepted Shadow's palace gate
system. A kernel driver hides an encrypted
maze inside. Navigate through it to extract
the credential for Shadow's internal network.

The palace gives NO feedback on whether
your moves succeed or hit a wall.

Or does it? The system is not as silent
as it seems. Five hidden flaws betray
the result of every move 欽?but each move
exposes only one of them.

Hint: after each reset, the first five
successful moves reveal each flaw exactly
once, in a fixed order.

[*] Connecting to Shadow gate driver...
[+] Gate module online.

[*] Maze grid: 13x13, Entry=(0,0), Exit=(12,12)
Commands:
W/A/S/D    - Navigate Up/Left/Down/Right
I/J/K/L    - Navigate Up/Left/Down/Right (alt)
R          - Reset to entry point
T          - Show operation log (position hidden)
H          - Show this help
Q / ESC    - Abort mission
```

- 迷宫大小: **13×13**
- 入口: **(0,0)** 左上角
- 出口: **(12,12)** 右下角

十、侧信道初步排查

10.1 排除 DbgPrint

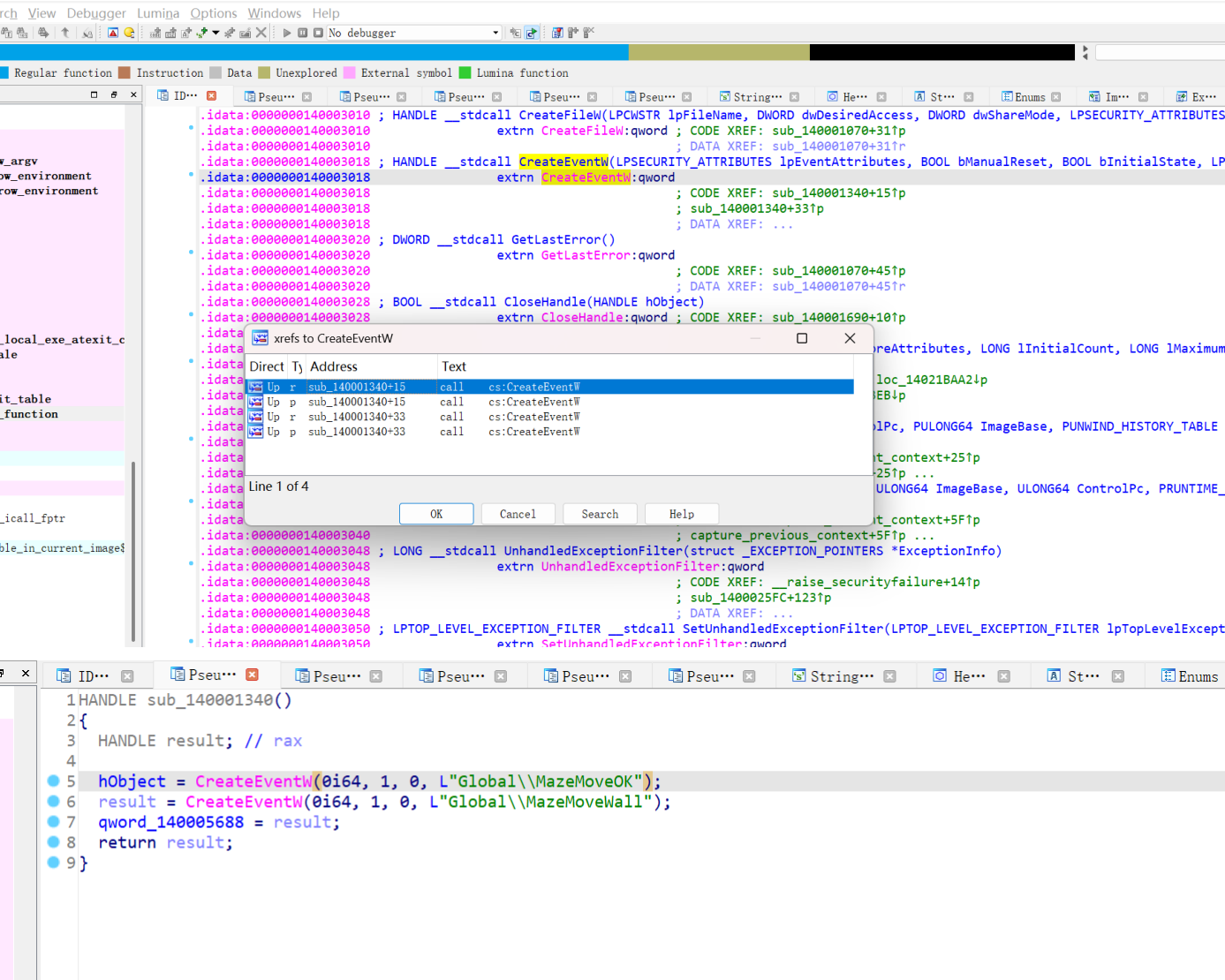
```
管理员: 命令提示符 - C:\Users\CTF\Desktop\ShadowGateApp.exe

[op #0] > r
[*] Reset to entry point.
[op #0] > s
[op #1] > d
[op #2] > s
[op #3] > w
[op #4] > d
[op #5] > a
[op #6] > i
[op #7] > j
[op #8] > k
[op #9] > l
[op #10] > t
[*] Operations sent: 10
[*] Sequence: DRDURLULDR
[op #10] > h
Commands:
W/A/S/D    - Navigate Up/Left/Down/Right
I/J/K/L    - Navigate Up/Left/Down/Right (alt)
R          - Reset to entry point
T          - Show operation log (position hidden)
H          - Show this help
Q / ESC    - Abort mission

[op #10] >
[?] Unknown command. 'h' for help.
[op #10] > □
[?] Unknown command. 'h' for help.
[op #10] > _
```

DbgView完全空白，确认 DbgPrint 不是侧信道。

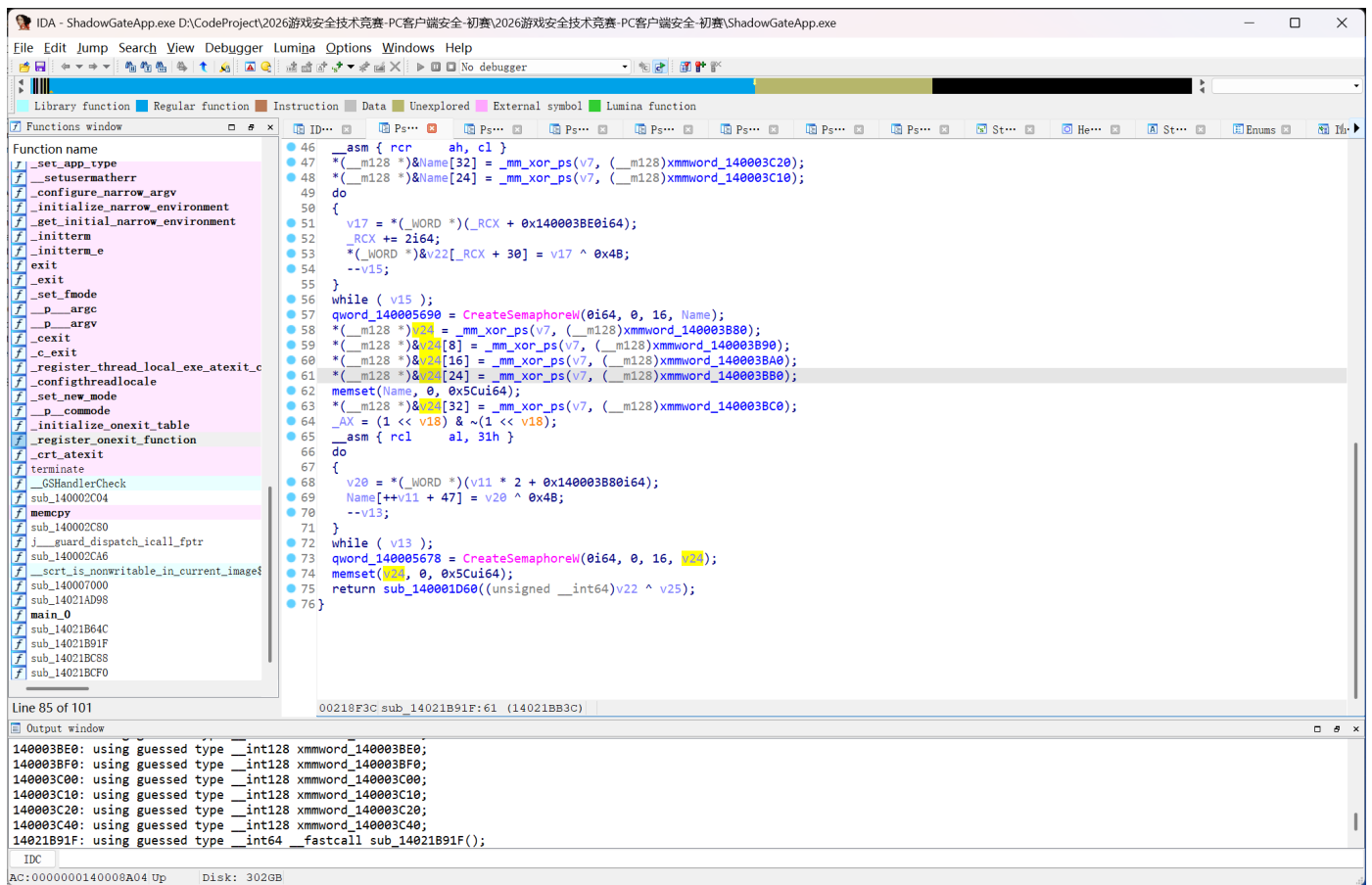
10.2 Event 侧信道线索



发现两个命名Event： Global\MazeMoveOK 和 Global\MazeMoveWall 。

10.3 Semaphore 侧信道线索

Semaphore名称在代码中被SSE XOR加密：

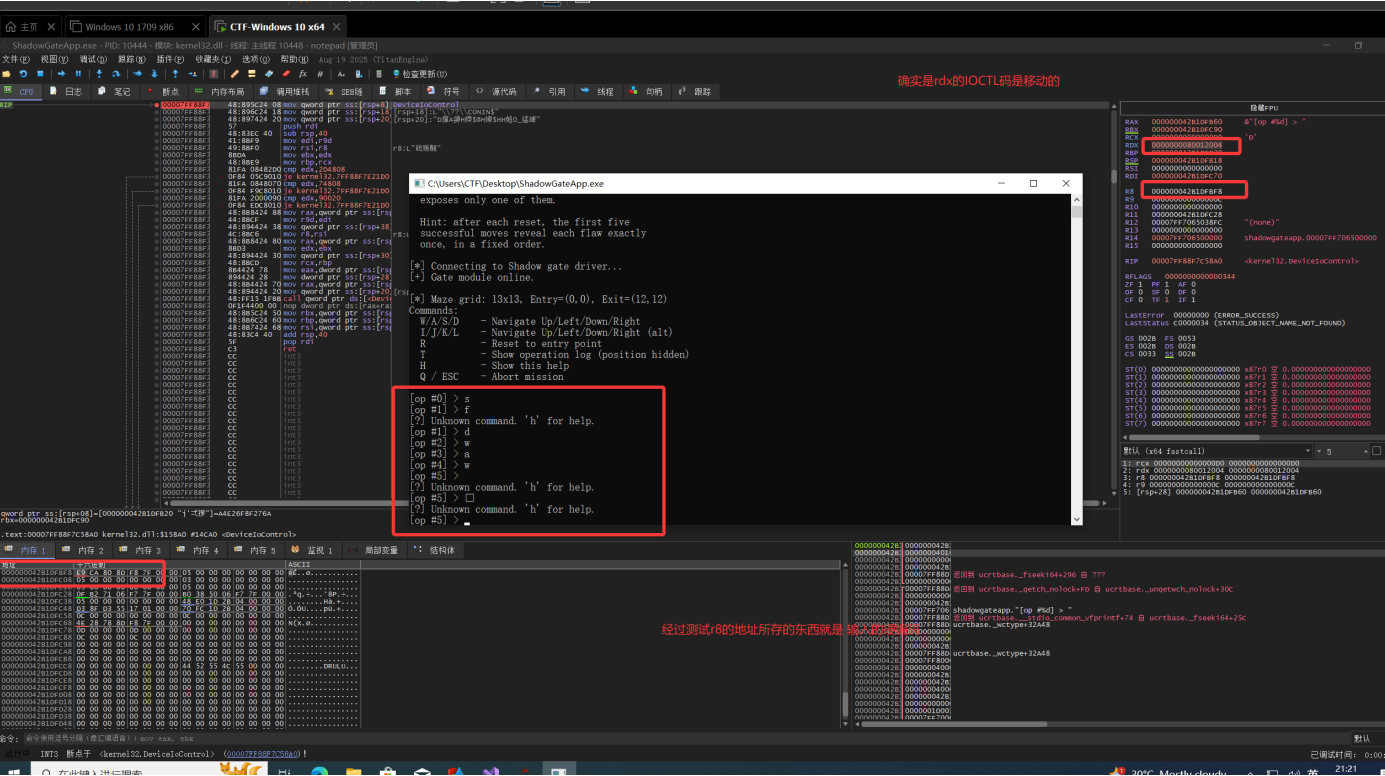


WinObjEx64发现两个可疑GUID格式Semaphore:

GLOBAL??	C:\ProgramData\Microsoft\Windows\Caches*...	Section	
RPC Control	C:\ProgramData\Microsoft\Windows\Caches*...	Section	
KnownDlls32	ComCatalogCache	Section	
BaseNamedOb	{B8E2C3D0-0F5A-5D9B-C6E7-2A3F4B5C6D7E}	Semaphore	
Restricted	{A7F3B2C1-9E4D-4C8A-B5D6-1F2E3A4B5C6D}	Semaphore	
UMDFCommuni	SM0:9992:304:WilStaging_02_p0h	Semaphore	
KnownDlls	SM0:9992:304:WilStaging_02_p0	Semaphore	
FileSystem	SM0:9908:304:WilStaging_02_p0h	Semaphore	
KernelObjects	SM0:9908:304:WilStaging_02_p0	Semaphore	
Callback	SM0:9908:120:WilError_03_p0h	Semaphore	
Security	SM0:9908:120:WilError_03_p0	Semaphore	
Device	SM0:9744:304:WilStaging_02_p0h	Semaphore	
Driver	SM0:9744:304:WilStaging_02_p0	Semaphore	
DriverStores	SM0:9744:120:WilError_03_p0h	Semaphore	
	SM0:9744:120:WilError_03_p0	Semaphore	
	SM0:964:304:WilStaging_02_p0h	Semaphore	
	SM0:964:304:WilStaging_02_p0	Semaphore	
	SM0:964:120:WilError_03_p0h	Semaphore	
	SM0:964:120:WilError_03_p0	Semaphore	
	SM0:9484:304:WilStaging_02_p0h	Semaphore	
	SM0:9484:304:WilStaging_02_p0	Semaphore	
	SM0:9332:304:WilStaging_02_p0h	Semaphore	
	SM0:9332:304:WilStaging_02_p0	Semaphore	
	SM0:9140:304:WilStaging_02_p0h	Semaphore	

- {B8E2C3D0-0F5A-5D9B-C6E7-2A3F4B5C6D7E}
- {A7F3B2C1-9E4D-4C8A-B5D6-1F2E3A4B5C6D}

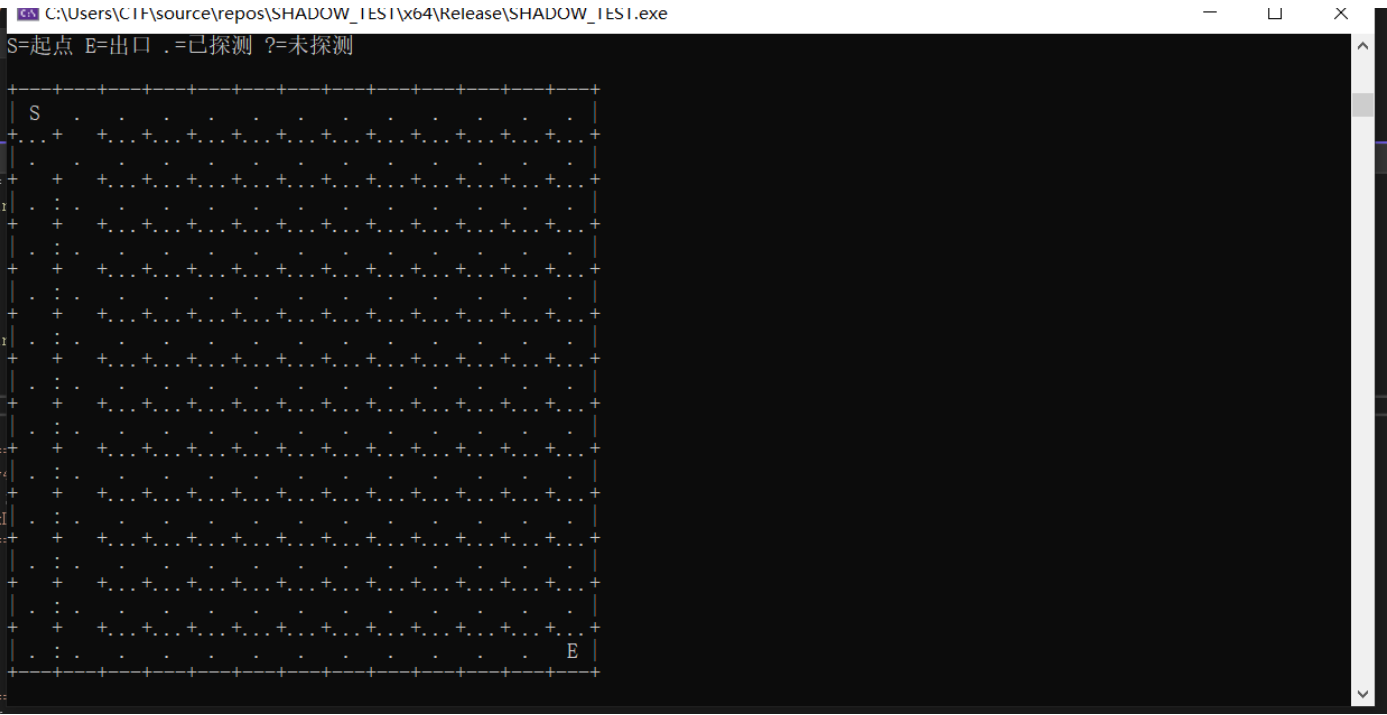
10.4 方向编码动态验证



键	方向	编码值
W	Up	0x52
A	Left	0x53
S	Down	0xD3
D	Right	0xD0

按键	方向	编码值
W	Up	0x52
A	Left	0x53
S	Down	0xD3
D	Right	0xD0

10.5 侧信道循环机制验证



Step #0 Event 100%可靠。Step #1~#4 Event不再触发。

十一、WinDbg内存Dump — 获取完整迷宫地图

11.1 操作步骤

通过WinDbg获取迷宫内存数据

思路

驱动在初始化时用 `ExAllocatePool2` 分配了472字节内存存储迷宫数据，并把指针存在全局变量 `P` 中。只要在WinDbg里找到 `P` 的运行时地址，读出指针值，再dump指针指向的内存，就能拿到完整迷宫数据，完全绕过侧信道。

第一步：IDA中定位全局指针P

在IDA中打开驱动 `ShadowGateSys.sys`，找到 `sub_140003208` 函数：

```
c
v0 = (void *)ExAllocatePool2(64164, 472164, 'ezaM');
P = v0;
```

双击变量 `P`，IDA跳转到它的定义位置，地址为 `0x1400050B8` (.data段)。

第二步：WinDbg中获取驱动基址

在WinDbg中断下虚拟机 (Ctrl+Break)，查看驱动信息：

```
2: kd> !drvobj \Driver\ShadowGate 2
```

从输出中找到 `DriverEntry` 的运行时地址：

```
DriverEntry: fffff80337168000
```

第三步：计算P的运行时地址

IDA中 DriverEntry 的地址是 `0x140008000` (IDA默认基址 `0x140000000` + 偏移 `0x8000`), 所以：

```
驱动运行时基址 = DriverEntry运行时地址 - DriverEntry偏移
                = ffffffff80337168000 - 0x8000
                = ffffffff80337160000
```

```
P的偏移 = IDA中P的地址 - IDA默认基址
          = 0x1400050B8 - 0x140000000
          = 0x50B8
```

```
P的运行时地址 = 基址 + 偏移
               = ffffffff80337160000 + 0x50B8
               = ffffffff803371650B8
```

第四步：读取P的指针值

```
2: kd> dq ffffffff803371650B8 L1
fffffff803`371650b8 fffffbb0c`517821c0
```

P指向的迷宫内存地址为 `fffffbb0c517821c0`。

地址计算：

```
DriverEntry运行时: ffffffff80337168000
IDA中DriverEntry:  0x140008000
驱动基址 = ffffffff80337168000 - 0x8000 = ffffffff80337160000
P运行时  = ffffffff80337160000 + 0x50B8 = ffffffff803371650B8
```

读取数据：

第五步：dump迷宫数据

472字节 = `0x1D8`：

```
2: kd> db fffffbb0c`517821c0 L1D8
```

前169字节 ($13 \times 13 = 169$) 就是迷宫地图，每字节0=通路，1=墙壁。后面是元数据（当前位置、移动记录等）。

关键公式总结

```
运行时地址 = (DriverEntry运行时地址 - IDA中DriverEntry偏移) + IDA中变量偏移
```

这个方法适用于任何需要在运行时读取驱动全局变量的场景，前提是驱动没有对.data段做额外的加密保护。

```
2: kd> dq fffff803371650B8 L1
fffff803`371650b8  fffffbb0c`517821c0

2: kd> db fffffbb0c`517821c0 L1D8
```

11.2 迷宫地图

S=起点 E=出口 .=通路 #=墙

	0	1	2	3	4	5	6	7	8	9	10	11	12
0:	S	.	.	.	.	.	.	#	.	.	.	.	.
1:	#	#	#	#	#	#	.	#	#	#	.	#	.
2:	.	.	.	.	.	#	.	.	.	.	.	#	.
3:	.	#	#	#	.	#	#	#	#	#	#	#	.
4:	.	#	.	.	.	.	.	.	.	.	.	#	.
5:	.	#	.	#	.	#	#	#	#	#	.	#	.
6:	.	#	.	#	.	#	.	.	.	#	.	#	.
7:	.	#	.	#	#	#	.	#	.	#	#	#	.
8:	.	#	.	.	.	.	.	#	.	#	.	.	.
9:	.	#	#	#	#	#	#	#	.	#	.	#	#
10:	.	.	.	#	.	.	.	#	.	#	.	#	.
11:	#	#	.	#	.	#	.	#	.	#	.	#	.
12:	.	.	.	.	.	#	.	#	.	.	.	.	E

十二、BFS最短路径求解

使用广度优先搜索（BFS）从(0,0)到(12,12)的最短路径。

	0	1	2	3	4	5	6	7	8	9	10	11	12
0:	S	.	.	.	.	.	.	#	.	.	.	.	.
1:	#	#	#	#	#	#	.	#	#	#	.	#	.
2:	.	.	.	.	.	#	.	.	.	.	.	#	.
3:	.	#	#	#	.	#	#	#	#	#	#	#	.
4:	.	#	.	.	.	.	.	.	.	.	.	#	.
5:	.	#	.	#	.	#	#	#	#	#	.	#	.
6:	.	#	.	#	.	#	.	.	.	#	.	#	.
7:	.	#	.	#	#	#	.	#	.	#	#	#	.
8:	.	#	.	.	.	.	.	#	.	#	.	.	.
9:	.	#	#	#	#	#	#	#	.	#	.	#	#
10:	.	.	.	#	.	.	.	#	.	#	.	#	.
11:	#	#	.	#	.	#	.	#	.	#	.	#	.
12:	.	.	.	.	.	#	.	#	.	.	.	.	E

最短路径长度：32步

*=路径 #=墙 .=通路

```

    0  1  2  3  4  5  6  7  8  9 10 11 12
0:  *  *  *  *  *  *  *  *  #  .  .  *  *  *
1:  #  #  #  #  #  #  *  #  #  #  *  #  *
2:  .  .  .  .  .  #  *  *  *  *  *  #  *
3:  .  #  #  #  .  #  #  #  #  #  #  #  *
4:  .  #  .  .  .  .  .  .  .  .  .  #  *
5:  .  #  .  #  .  #  #  #  #  #  .  #  *
6:  .  #  .  #  .  #  .  .  .  #  .  #  *
7:  .  #  .  #  #  #  .  #  .  #  #  #  *
8:  .  #  .  .  .  .  .  #  .  #  *  *  *
9:  .  #  #  #  #  #  #  #  #  .  #  *  #  #
10: .  .  .  #  .  .  .  #  .  #  *  #  .
11: #  #  .  #  .  #  .  #  .  #  *  #  .
12: .  .  .  .  .  #  .  #  .  .  *  *  *
```

方向序列: RRRRRRDDRRRRUURDDDDDDDDLLDDDDRR

WASD序列: DDDDDSSDDDDWDDSSSSSSSAASSSSDD

十三、Flag获取

```
=====
ACCESS GRANTED - Credential extracted!
=====
[CREDENTIAL] flag{SHAD0WNT_HYPERVMX}
[*] Your input (32 ops sent): RRRRRRDDRRRRUURDDDDDDDDLLDDDDRR
```

Flag: flag{SHAD0WNT_HYPERVMX}

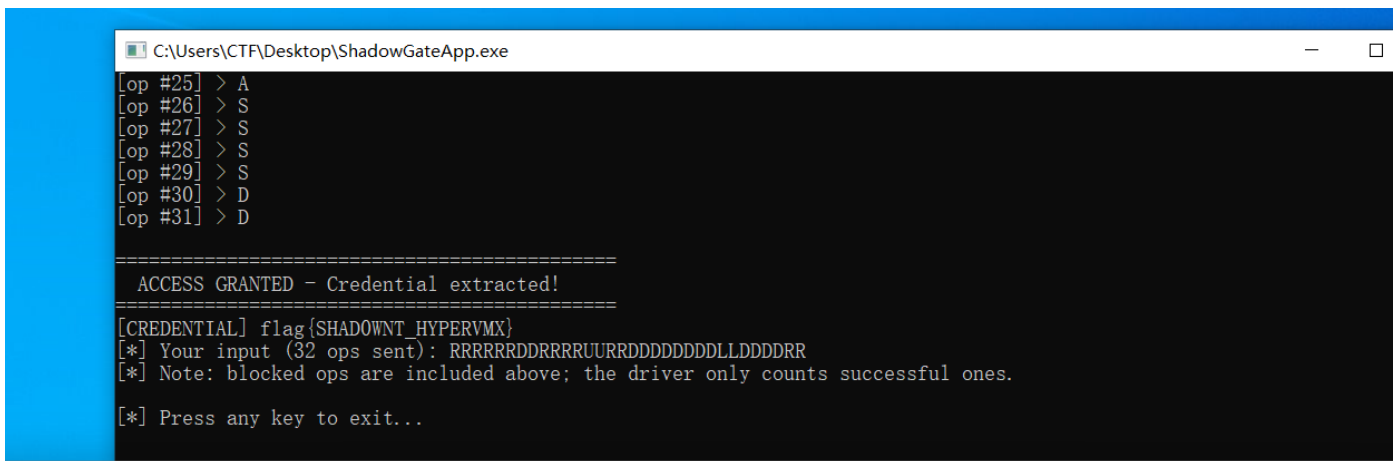
十四、自动化工具开发 — 逆向Move函数与Flag提取

14.1 突破点选择：字符串交叉引用

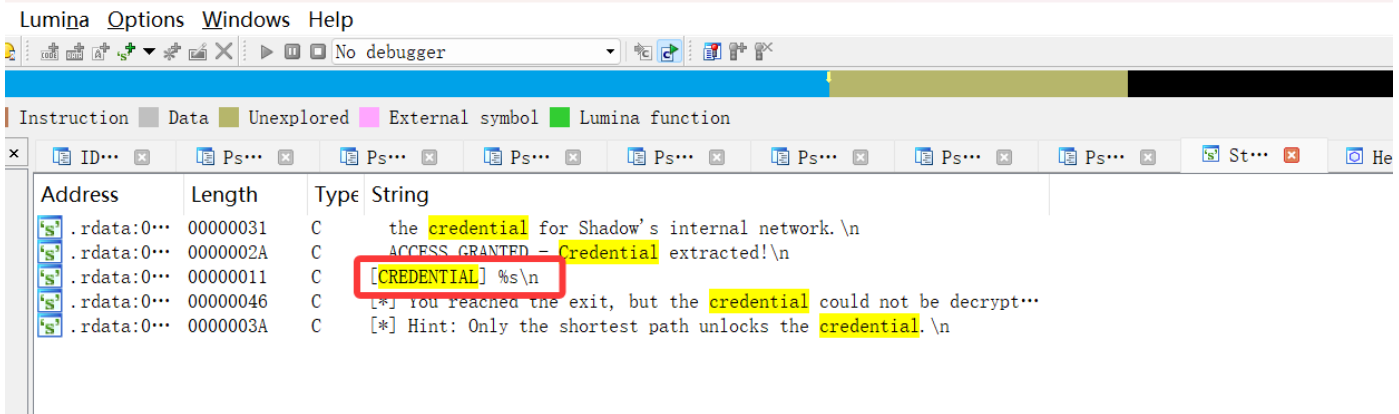
在IDA字符串窗口搜索 CREDENTIAL 和 ACCESS GRANTED , 所有字符串按X查找交叉引用时IDA报

There are no xrefs ——exe被重度混淆。

改用x64dbg字符串搜索（当前模块→String references）, 找到 [CREDENTIAL] %s\n 的引用位置。

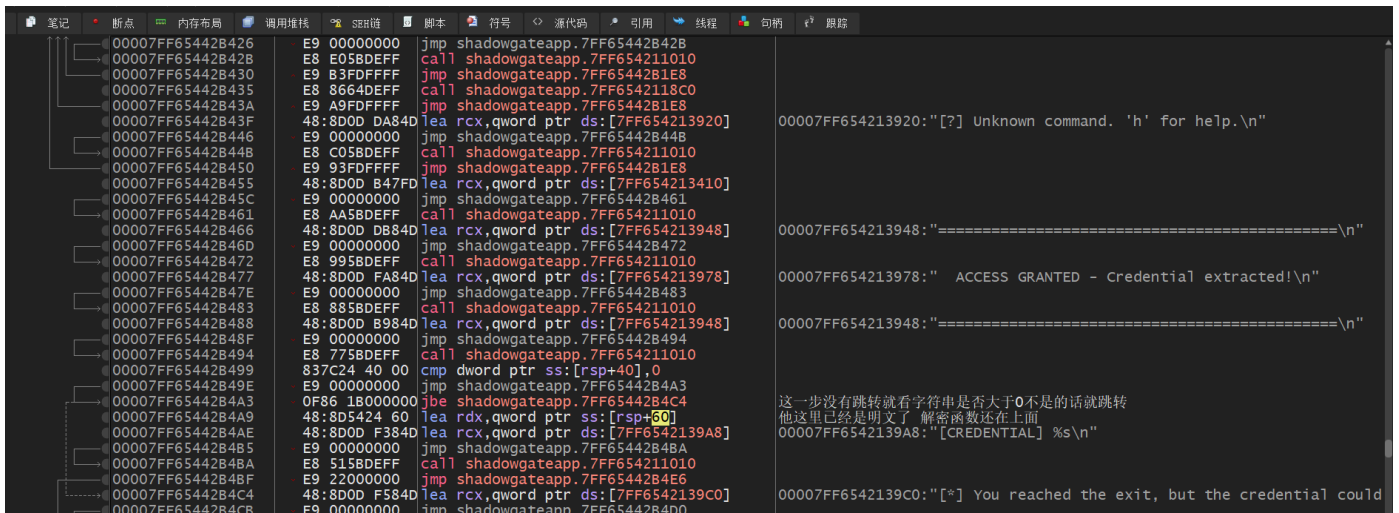


2026游戏安全技术竞赛-PC客户端安全-初赛\2026游戏安全技术竞赛-PC客户端安全-初赛\ShadowGateApp.exe



14.2 主循环反汇编分析

从 [CREDENTIAL] 引用位置往上追溯，完整还原主循环逻辑。



Move函数原型（逆向还原）：

```
int Move(HANDLE hDevice, // RCX
        __int64 rawDir, // RDX - 原始方向值
        int opCount, // R8 - 操作计数
        char* credBuf, // R9 - credential输出
        DWORD* credLen); // [RSP+20] - 长度输出
// 返回: 0=正常移动, 1=到达终点, -1=IOCTL失败
```

键盘到方向值映射（主循环switch跳表还原）：

```
mov dl, 0x10 / mov al, 'U' → W键 (Up)
mov dl, 0x20 / mov al, 'D' → S键 (Down)
mov dl, 0x30 / mov al, 'L' → A键 (Left)
mov dl, 0x40 / mov al, 'R' → D键 (Right)
```

14.3 Move函数内部 (sub_14021B64C)

方向编码算法：

```
v6 = _RDX ^ 0xFA;
LOBYTE(_RDX) = (uint8)(_RDX ^ 0x40) >> 5;
v8 = (uint8)(_RDX | (8 * v6));

// RDX=0x10(Up)→0x52 RDX=0x20(Down)→0xD3
// RDX=0x30(Left)→0x53 RDX=0x40(Right)→0xD0
```

IOCTL通信：

```
v19 = (uint8)v8; // 编码后方向
v20 = a3 ^ v8 ^ 0xDEAD1337; // 校验值
DeviceIoControl(hDevice, 0x80012004, &v19, 0x0C, OutBuffer, 0x84, ...);
```

关键纠错 — "WIN!" 不是 "WALL":

```
if (HIDWORD(v15) == 0x57494E21) { // 1464421921 = 0x57494E21 = "WIN!"
    memcpy(credBuf, Src, Size); // 直接拷贝credential明文
}
```

之前错算为 0x57414C41 ("WALL")。实际是通关成功标记 "WIN!"。

14.4 自动化工具迭代过程

v1 — 独立进程直接发IOCTL： 32步全OK但credential为0。

双重编码Bug： x64dbg抓的0xD0是编码后的v8，不是原始RDX。二次编码变成0x54，驱动不认。Step 0的Event:WALL证实了撞墙。

76 SHADOW TEST.exe 生命周期事件 线程: 堆栈帧:

C:\Users\CTF\source\repos\SHADOW_TEST\x64\Release\SHADOW_TEST.exe

Step 28: Down (raw=0xD3, encoded=0x4C, chk=0xDEAD1367) OK 132 bytes
Step 29: Down (raw=0xD3, encoded=0x4C, chk=0xDEAD1366) OK 132 bytes
Step 30: Right (raw=0xD0, encoded=0x54, chk=0xDEAD137D) OK 132 bytes
Step 31: Right (raw=0xD0, encoded=0x54, chk=0xDEAD137C) OK 132 bytes

[+] 统计: 总步数=32, 撞墙=0, 成功移动=32

[+] 最后一步返回数据分析:
WALL标记 [0x3C]: 0x00000000 (终点)
Credential长度 [0x80]: 0
原始数据 hex dump:
8D 7D A4 BA 58 14 5D E8 D9 F4 C9 68 C2 67 7F CA
E4 7D DB 7C 52 55 48 B8 A9 7F 69 39 20 13 0C BA
53 8E CF 33 CE 70 90 D3 5C 96 96 BC 0E 57 81 81
0A 61 F2 60 19 52 F9 3F EE 16 42 68 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00

[*] 到达终点, 但credential长度为0
[*] Hint: Only the shortest path unlocks the credential.

[+] 完成, 按任意键退出

逐步排查:

尝试	操作	结果
创建Event	CreateEventW(MazeMoveOK/Wall)	Event:OK确认, credential仍为0
Query IOCTL	先查询迷宫参数	正常返回, credential仍为0
重命名exe	改名ShadowGateApp.exe	无效
DLL注入复用句柄	从全局变量读hDevice	同进程同句柄, credential为0
线程劫持	修改主线程RIP	同TID, credential仍为0

14.5 IAT Hook抓包 — 发现调用栈验证

Hook DeviceIoControl 被动记录数据, 手动走32步:

发现1 — 输出buffer差异:

来源	DWORD[0]	含义
控制台exe	0xBAAD9Fxx	真实数据
我们的DLL	0xBAA0C0DE	"BAD CODE"假数据

发现2 — credential是明文:

偏移0x40处直接就是 flag{SHAD0WNT_HYPERVMX} 的ASCII明文。驱动直接返回, exe无解密。

发现3 — 调用栈验证：

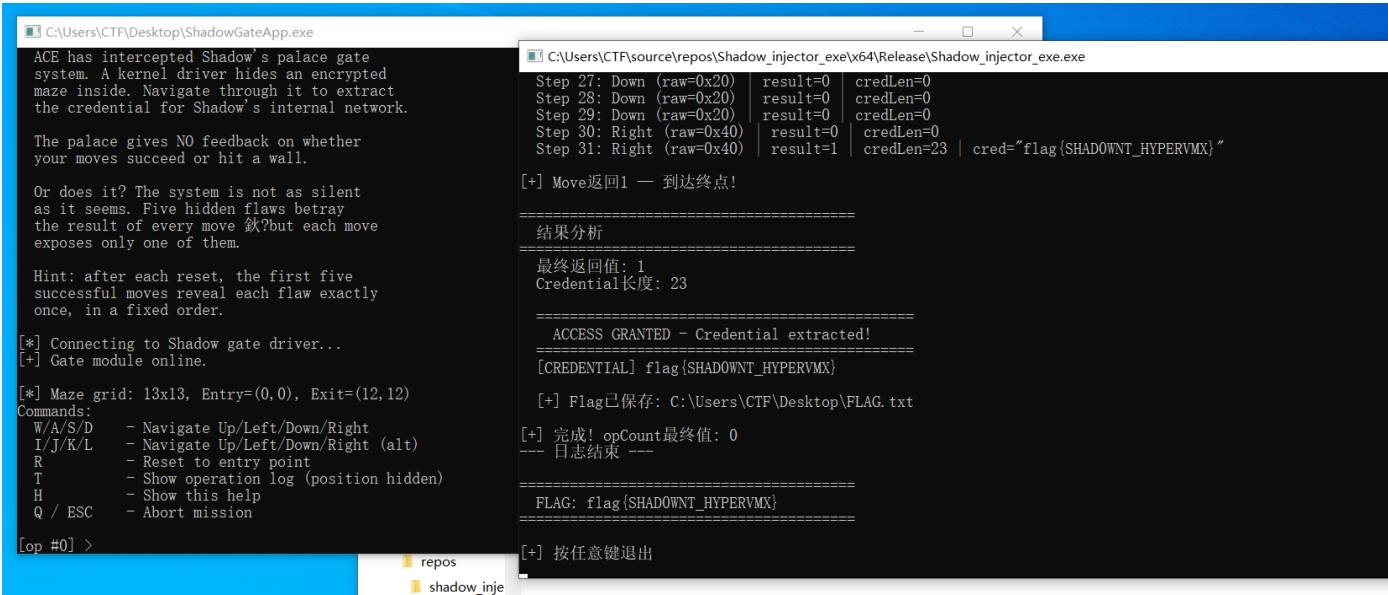
驱动通过 ZwQueryVirtualMemory 检查 DeviceIoControl 返回地址。只有在exe的 .text 段内才返回真数据。

14.6 最终方案：直接调用exe内部函数

通过函数指针调用exe的Move函数，调用栈经过exe的 .text 段：

DLL代码 → exe的Move函数(sub_14021B64C) → DeviceIoControl → 驱动验证通过

运行结果：



Step 31: Right (raw=0x40) | result=1 | credLen=23 | cred="flag{SHAD0WNT_HYPERVMX}"

=====

ACCESS GRANTED - Credential extracted!

=====

[CREDENTIAL] flag{SHAD0WNT_HYPERVMX}

14.7 驱动反分析机制清单

机制	描述
代码混淆	VMProtect级别混淆，花指令+不透明谓词
自定义DeviceType	用0x8001替代标准0x22，干扰IOCTL扫描
方向编码	三层XOR+移位
调用栈验证	ZwQueryVirtualMemory检查返回地址所属模块
假数据返回	非法调用者收到0xBAA0C0DE
反调试	IsDebuggerPresent定期检查，软断点触发自杀
字符串混淆	所有关键字字符串无IDA交叉引用

十五、侧信道检测 — 全部五种发现与验证

15.1 机制概述

控制台启动提示：

"Five hidden flaws betray the result of every move — but each move exposes only one of them. Hint: after each reset, the first five successful moves reveal each flaw exactly once, in a fixed order."

规则：Reset后前5次成功移动按固定顺序各暴露1种侧信道。撞墙不推进计数器。5种侧信道循环使用。

15.2 检测工具架构

所有检测工具采用DLL注入 + 调用exe内部函数的架构（绕过调用栈验证）。在每次Move前后快照多维状态做差分对比，经过多轮迭代最终覆盖了以下检测维度：

Event信号、Semaphore计数、高精度计时、PEB快照、VirtualQuery内存属性、.data段内容diff、线程/进程优先级、CPU亲和性、Debug寄存器(DR0-DR7)、进程句柄数、TEB快照、GetLastError、.g-:: 段采样。

15.3 侧信道 #0 — Event对象

检测方法： WaitForSingleObject 检查 Global\MazeMoveOK / Global\MazeMoveWall

验证数据（4轮全部一致）：

全通行: Step 0: MazeMoveOK ★ Step 1: 无Event
全撞墙: Step 0-7: 全部MazeMoveWall ★ (计数器不推进)
交替: Step 0通行: OK ★ Step 1撞墙: WALL ★ (仍在#0)
复验: Step 0: MazeMoveOK ★ (顺序固定)

15.4 侧信道 #1 — Semaphore计数

Semaphore对象 (名称在exe中被SSE XOR加密, 通过WinObjEx64发现) :

- {A7F3B2C1-9E4D-4C8A-B5D6-1F2E3A4B5C6D} — 通行时+1
- {B8E2C3D0-0F5A-5D9B-C6E7-2A3F4B5C6D7E} — 撞墙时+1

检测方法: NtQuerySystemInformation 枚举进程句柄 + NtQuerySemaphore 读计数

验证数据:

全通行: Step 1: {A7F3B2C1} 0→1 ★
交替: Step 2(第2次成功): {A7F3B2C1} +1 ★ (通行)
Step 3(撞墙仍#1): {B8E2C3D0} +1 ★ (撞墙用另一个)

15.5 侧信道 #2 — TEB LastError 正向

这是整个分析过程中最难找的部分。在排除了PEB、.data内容、线程优先级、进程优先级、CPU亲和性、IOCTL返回数据模式、VirtualQuery保护级别、Debug寄存器(DR0-DR7)、进程句柄数之后, 一度以为step 2和step 3的侧信道只能从内核态观测。

转机出现在扩展TEB检测维度的时候——之前的检测器只查了PEB (进程环境块), 没查TEB (线程环境块)。驱动通过 KeStackAttachProcess 附加到目标进程后, 直接写入当前线程TEB偏移

0x68处的 LastErrorValue 字段。加上一个简单的 SetLastError(0) + GetLastError() 后, Step 2通行时跳出了 0xC0DE0001。

检测方法:

```
SetLastError(0);           // 清零
DoMove(...);              // 执行移动
DWORD err = GetLastError(); // 读取
if (err == 0xC0DE0001) → 通行
if (err == 0)             → 撞墙 (LastError未改变)
```

精确验证数据:

```
Step 2 通行: LastError = 0x00000000 → 0xC0DE0001 ★  
TEB[0x68]: 01 00 DE C0
```

```
Step 2 撞墙: LastError = 0x00000000 → 0x00000000 ★ 不变  
TEB[0x68]: 00 00 00 00
```

驱动通过 `KeStackAttachProcess` 附加进程后直接写 `TEB+0x68`。通行写 `0xC0DE0001` ("CODE TRUE")，撞墙不写。

15.6 侧信道 #3 — TEB LastError 反向

找到#2之后，顺藤摸瓜验证#3——果然也是TEB LastError，但逻辑完全反转。

检测方法： 与#2相同，`SetLastError(0) + GetLastError()`

精确验证数据：

```
Step 3 通行: LastError = 0x00000000 → 0x00000000 ★ 不变 = 通行  
TEB[0x68]: 00 00 00 00
```

```
Step 3 撞墙: LastError = 0x00000000 → 0xC0DE0000 ★  
TEB[0x68]: 00 00 DE C0
```

#2和#3互为补充——#2通行有信号、撞墙无信号；#3通行无信号、撞墙有信号。两者组合可以做交叉验证。出题人用了 `0xC0DE0001` (TRUE) 和 `0xC0DE0000` (FALSE) 作为标记值，算是一个小彩蛋。

15.7 侧信道 #4 — 内存保护级别变化

检测方法： `VirtualQuery(exeBase + 0x5000)` 检测Protect属性

验证数据：

```
全通行 Step 4: exe .data段(+0x5000)  
READWRITE(0x4) → EXECUTE_READWRITE(0x40) ★
```

WinDbg验证：

```
bp /p <EPROCESS> nt!NtProtectVirtualMemory  
→ 只在Step 4(第5次成功移动)触发，其他步不触发
```

驱动通过 KeStackAttachProcess 附加进程后使用 NtProtectVirtualMemory 修改exe .data 段的页保护属性。

15.8 寻找 #2/#3 的排除过程

在最终找到TEB LastError之前，完整的排除记录：

综合检测器排除的方向（Step 2/3全部无变化）：

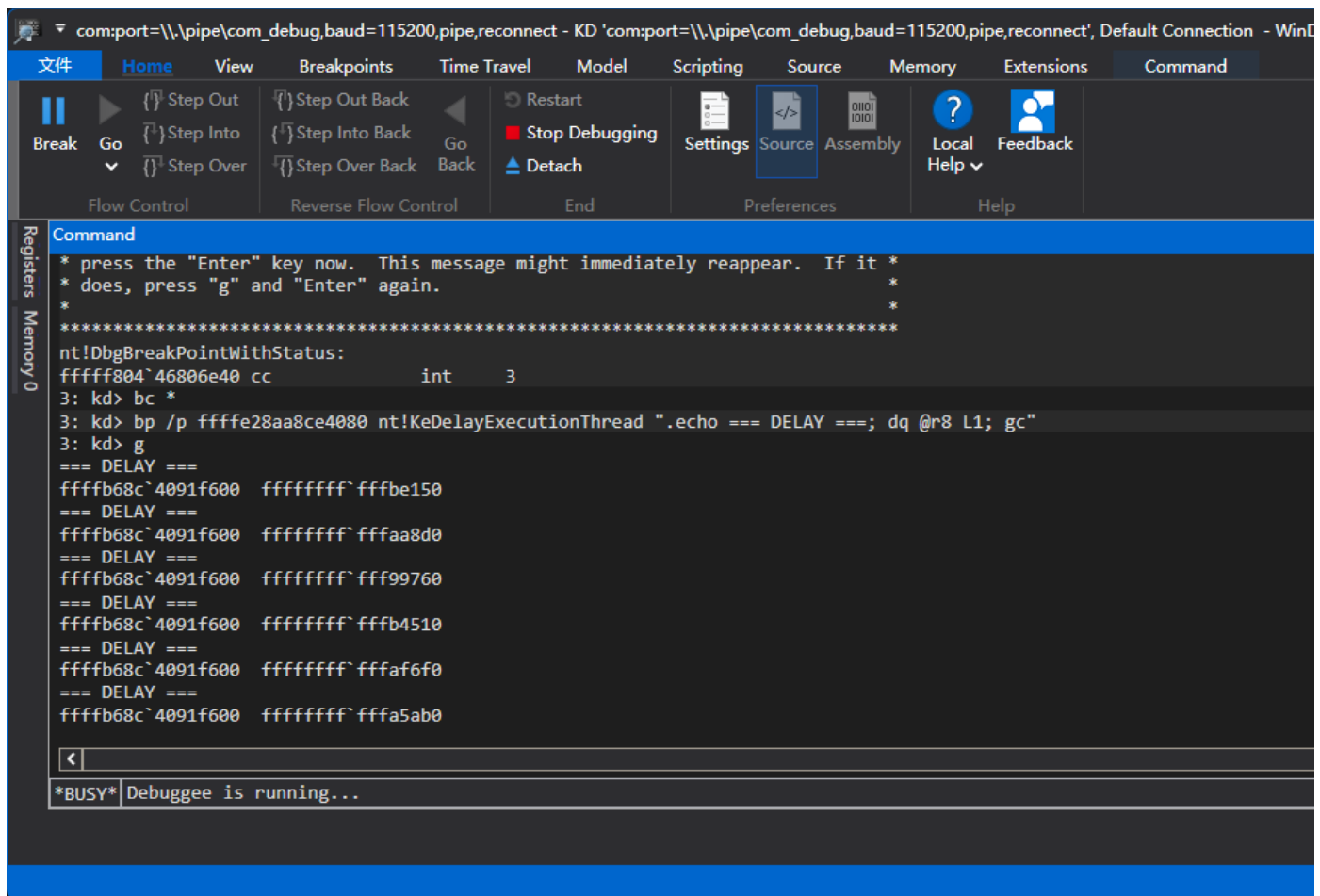
检测项	API	结果
PEB快照（8KB逐字节）	直接内存对比	无变化
exe .data段内容（1712字节）	逐字节memcmp	无变化
线程优先级	GetThreadPriority	始终=0
进程优先级	GetPriorityClass	始终=0x20
CPU亲和性	GetProcessAffinityMask	始终=0xF
IOCTL返回数据DWORD[0]模式	Hook抓包对比	通行/撞墙无规律
VirtualQuery保护级别	VirtualQuery	Step 2/3无变化
Debug寄存器(DR0-DR7)	GetThreadContext	DR0=0, DR7=0不变
进程句柄数	GetProcessHandleCount	54→54不变
.g-:: 段采样（前4KB+后4KB）	逐字节memcmp	无变化

WinDbg内核断点分析 KeDelayExecutionThread：

```

C:\Users\CTF\Desktop\ShadowGateApp.exe
[*] Reset to entry point.
[op #0] > d
[op #1] > d
[op #2] > d
[op #3] > w
[op #4] > d
[op #5] > w
[op #6] > r
[*] Reset to entry point.
[op #0] > d
[op #1] > d
[op #2] > d
[op #3] > d
[op #4] > d
[op #5] > d
[op #6] > r
[*] Reset to entry point.
[op #0] > f
[?] Unknown command. 'h' for help.
[op #0] > r
[*] Reset to entry point.
[op #0] > d
[op #1] > d
[op #2] > d
[op #3] > r
[*] Reset to entry point.
[op #0] > d
[op #1] > d
[op #2] > w
[op #3] >

```



每步都调用，延迟值每次不同，无法用于区分通行/撞墙。

蓝屏问题： 高频统计测试（数百次Reset+Move）触发BSOD两次。后续将Reset总数控制在25次以内。

逆驱动（失败）： IOCTL handler在VMProtect混淆区域，花指令全是垃圾数据，静态不可行。

15.9 实时可视化监控工具

确认全部5种侧信道后，开发了实时可视化监控工具。通过IAT Hook被动监控 DeviceIoControl ，在控制台内实时显示每一步的侧信道触发状态：

Step	Dir	Ch#	#0 Event	#1 Sem	#2 TEB+	#3 TEB-	#4 Mem	Result
1	Right	#0	OK *	--	--	--	--	PASS
2	Right	#1	--	OK +1	--	--	--	PASS
3	Right	#2	--	--	C0DE01	--	--	PASS
4	Right	#3	--	--	--	--	--	PASS
5	Right	#4	--	--	--	--	RW>RWX	PASS
6	Right	#0	--	--	--	--	--	PASS
7	Down	#1	OK *	--	--	--	--	PASS
8	Down	#2	--	--	C0DE01	--	--	PASS

工具跟踪成功移动计数器（ successCount % 5 ），用对应的判定逻辑区分PASS/WALL。绿色=触发，灰色=未触发，红色=WALL。按R清屏重置。纯被动监控零蓝屏风险。

15.10 五种侧信道完整汇总

#	侧信道类型	通行信号	撞墙信号	检测API	驱动机制
# 0	Event对象	MazeMove OK Signal	MazeMove Wall Signal	WaitForSingleObject	ZwSetEvent
# 1	Semaphore计数	{A7F3B2C1} +1	{B8E2C3D0} +1	NtQuerySemaphore	KeReleaseSemaphore
# 2	TEB LastError(正向)	0xC0DE0001	不变(0)	GetLastError	KeStackAttachProcess + 写TEB+0x68
# 3	TEB LastError(反向)	不变(0)	0xC0DE0000	GetLastError	KeStackAttachProcess + 写TEB+0x68
# 4	内存保护级别	RW→RWX	不变	VirtualQuery	NtProtectVirtualMemory

十六、提交文件清单

源码工程

所有源码位于 ..\测试代码\ , 共6个Visual Studio 2022工程:

VS工程文件夹	核心源码	功能
shadow_inject/	dllmain.cpp	自动化求解 (DLL注入+调用exe内部函数提取Flag)
shadow_injector/	dllmain.cpp	IOCTL数据监控 (IAT Hook DeviceIoControl 被动记录)
Shadow_injector_exe/	Shadow_injector_exe.cpp	DLL注入器 (CreateRemoteThread + LoadLibraryW)
SHADOW_TEST/	SHADOW_TEST.cpp	独立迷宫求解器 (BFS+IOCTL+方向编码还原)
sidechannel_detector/	dllmain.cpp	五种侧信道检测验证工具 (自动化通行/撞墙对比)
visualizer/	dllmain.cpp	侧信道实时可视化监控 (控制台内彩色表格显示)

DLL工程中的 framework.h 、 pch.h 、 pch.cpp 为VS自动生成的预编译头文件, 核心代码全部在 dllmain.cpp 中。

编译产物

所有编译产物集中存放在 Shadow_injector_exe/x64/Release/ 目录:

文件	来源工程	功能
Shadow_injector_exe.exe	Shadow_injector_exe	DLL注入器
shadow_inject_hook取flag.dll	shadow_inject	自动化Flag提取
shadow_inject_Hook取每一次返回的数据.dll	shadow_injector	IOCTL数据抓包
shadow_inject_side_channel_detector.dll	sidechannel_detector	五种侧信道验证
shadow_inject_可视化工具.dll	visualizer	实时可视化监控
shadow_inject_测信道蓝屏.dll	—	时间侧信道分析 (⚠️有BSOD风险)

使用时将目标DLL重命名为 shadow_inject.dll 放在注入器同目录，管理员运行

Shadow_injector_exe.exe 注入。

附录：关键公式

IOCTL码 (自定义DeviceType 0x8001) : Move=0x80012004, Reset=0x80012008, Query=0x8001200C

方向编码: 原始值 Up=0x10, Down=0x20, Left=0x30, Right=0x40 → 编码后 0x52, 0xD3, 0x53, 0xD0

编码算法: v6=RDX^0xFA; rdx_low=(RDX^0x40)>>5; v8=rdx_low|(8*v6)

IOCTL输入(12字节): [编码方向(8B)] [opCount^dir^0xDEAD1337(4B)]

IOCTL输出(132字节):

[加密数据(60B)] [WIN标记0x57494E21(4B)] [credential明文(64B)] [长度(4B)]

TEB侧信道标记值: 0xC0DE0001 = 通行(TRUE), 0xC0DE0000 = 撞墙(FALSE)